HUSH: Provably Safe Sub-Block Hibernation for Real-Time Event-Driven SNN Inference on FPGAs

Anubhav Jha and Rajat Subhra Chakraborty
Indian Institute of Technology Kharagpur, India

***Abstract*—In a fully connected spiking layer, most of the time and energy goes into fetching the weights of every neuron for every input spike, although few neurons fire. We present HUSH, an output-side skipping scheme that puts a sub-block of eight neurons to sleep for a timestep only when a cheap integer envelope proves that none of them can reach the threshold. At every timestep the gated layer therefore emits exactly the spikes that the ungated update from its current state would emit; only the state of sleeping neurons evolves approximately. The envelope combines the exact maximum membrane potential of the sub-block with a per-input proxy table and is computed while the previous timestep is processed; its envelope and lane arithmetic is verified formally at the implementation's bit widths. Trained with the gate in the loop, a 578-256-10 integer network on N-MNIST runs on a Xilinx Artix-7 XC7A100T at 100 MHz entirely on chip, from raw sensor events to the class: at 95.74 % accuracy it is 2.44× faster than the same hardware with the gate off (115 µs vs. 280 µs per inference; 98.21 % accuracy) with 57 % less active energy per inference (18.4 vs. 42.5 µJ, estimated from gate-level switching activity), and at 96.54 % 1.75× faster with 44 % less; the gate costs 79 % more LUTs and 19 % more block RAM. On the same chip, with BASE's 10-epoch weights, the open Spiker+ generator needs 818 µs and 255 µJ (6.0× BASE). In cycle-accurate RTL replay of four N-MNIST recordings at their recorded timing, every 3 ms window is processed within 4.5 µs of its end, and the design sustains up to 1.33–2× the event rate of the ungated design with the same 1,024-event input buffer.**

I. Introduction

Event-driven spiking neural networks (SNNs) are attractive for always-on sensing because their input is sparse and their arithmetic is additive. On FPGAs, accelerators usually exploit *input* sparsity: work is done only for input spikes that occur. For a fully connected layer with N neurons, each input spike still costs N weight reads from block RAM, although in a given timestep only a few percent of the neurons fire. Skipping the *output* side, i.e. neurons that will not fire, is the remaining large saving, but it is hard: whether a neuron fires depends on the very weights one wants to skip. The output-side schemes published for SNNs are, to our knowledge, heuristics (they predict that a neuron stays silent and can be wrong) and therefore change the network's behaviour in ways that are hard to bound.

We make output-side skipping *provably safe*. Our contributions:

1) **HUSH gate:** an integer envelope per sub-block of eight neurons that upper-bounds every member's next membrane potential from the sub-block's exact current maximum and a per-input proxy table; a sub-block sleeps only if the bound is below threshold (Theorem 1), so at every timestep the gated layer emits exactly the spikes of the ungated update from its current state. We separate this per-timestep guarantee from end-to-end equivalence, which it does not give, and measure the difference. The RTL envelope and lane arithmetic is proved, at its real bit widths and for all values, to keep the bound, given that each neuron's drive is at most the proxy drive D_c (true by construction of the proxy table).
2) **Architecture:** an envelope unit overlapped with the processing of the previous timestep, per-sub-block weight-column RAM enables, a whole network on chip (hidden layer, output layer overlapped one timestep behind, and an integer class decision), and a stream front-end that accepts raw DVS events.
3) **Evaluation on one chip:** bit- and cycle-exact RTL against a golden model; gate-level activity-based power; the same RTL with the gate off as baseline (BASE); Spiker+ [1] on the same device with BASE's weights; three training seeds per point; and real-time operation on N-MNIST event streams replayed at sensor timing.

II. Background and related work

Integer LIF. Each neuron keeps a 16-bit saturating potential V. Per timestep t, $U = \mathrm{sat}(L(V) + \sum_{j \in A_t} w_j)$ with leak $L(V) = V - (V \gg k)$, $k = 4$, INT8 weights and A_t the active inputs; the sum is exact before the single saturation. It spikes, $s = [U \geq \theta]$ ($\theta = 64$), and keeps $V' = U - \theta s$ (soft reset).

FPGA SNN accelerators. Spiker [2], its generator Spiker+ [1] and FireFly [3] are clock-driven: every neuron is updated every timestep. Table VII lists the designs we found that report N-MNIST results on an FPGA. They differ in network, device and power method; for two we found no latency.

Output-side skipping. Three recent designs skip work for neurons predicted not to fire: Early [4] stops accumulation for neurons whose partial sum of the largest weights stays below a tuned threshold; SITRA [5] scores each output of a spiking transformer with a weight-MSB dot product and skips the weight fetches of outputs it predicts silent, overlapping detection with the previous token; PS4 [6] speculates whether each neuron fires within a time window and recomputes the window step by step when a speculation fails. Early and SITRA accept mispredictions: Early reports a 0.54 % mean accuracy loss over its benchmarks and SITRA 1–2 %; PS4

trades mispredictions for recomputation time instead. All three use tuned thresholds or speculation and are evaluated with architecture simulators and ASIC synthesis (28 or 65 nm), not on an FPGA. SPARSH [7] skips zero weights, zero spikes and refractory neurons; C-STEP [8] and the early-stop of [9] end inference early in time. For ReLU CNNs, SnaPEA [10] has an exact mode: with non-negative inputs and weights sorted by sign, a convolution stops once its partial sum turns negative. It still reads weights up to that point, per neuron, and has no state across time. HUSH differs in that its prediction is a proven bound that is evaluated before any weight of the sub-block is read: a sleeping sub-block cannot contain a neuron that would fire at that timestep, the bound carries membrane state across timesteps, and its granularity matches a block-RAM column, so a skip removes whole memory reads on an FPGA.

III. The HUSH gate

Neurons are grouped into sub-blocks c of $b = 8$ (eight INT8 weights fill one 64-bit block-RAM word; other sizes were not explored). Two tables are derived offline from the weights: the proxy $P_{jc} = \max_{i \in c} W_{ji}$ and the mean $Q_{jc} = \lfloor \mathrm{mean}_{i \in c} W_{ji} \rfloor$. The hardware keeps $R_c = \max_{i \in c} V_i$ exactly after every timestep. Before the weights of timestep t are fetched, the envelope unit accumulates $D_c = \sum_{j \in A_t} P_{jc}$ and evaluates

$$E_c = \mathrm{sat}\big(L(R_c) + \max(0, D_c)\big), \qquad g_c = [E_c \geq \theta]. \quad (1)$$

Theorem 1 (Safety). *If $g_c = 0$, every neuron i in sub-block c has $U_i < \theta$ in the ungated update from the same state.*

Proof. L is monotone, so $L(V_i) \leq L(R_c)$; $W_{ji} \leq P_{jc}$ gives $\sum_{j \in A_t} W_{ji} \leq D_c \leq \max(0, D_c)$; and sat is monotone. Hence $U_i \leq E_c < \theta$. □

A sleeping sub-block skips its weight reads. Its neurons are updated with a shared mean-field drive $q_c = \sum_{j \in A_t} Q_{jc}$, $U_i = \mathrm{sat}(L(V_i) + q_c) \leq E_c < \theta$ (since $Q_{jc} \leq P_{jc}$), so they cannot spike either, and R_c stays exact.

What is guaranteed and what is not. Two properties must be kept apart (Fig. 1). (A) *Per-timestep safety*: from HUSH's own state V^{H}_{t-1}, the gated update emits exactly the spikes of the ungated update, since awake sub-blocks are computed exactly and sleeping ones contain no neuron that would reach θ. This holds at every timestep, by Theorem 1. (B) *Trajectory equivalence* with an ungated run, $V^{\mathrm{H}}_t = V^{\mathrm{B}}_t$, is *not* guaranteed: a sleeping neuron receives the mean-field drive instead of its own, so its state, and later spikes, can differ. We measured both on 1,000 test samples per point: the golden model's shadow update finds no spike that HUSH suppresses relative to (A). (B) is far from exact: run without the gate, the same weights emit 553 instead of 304 hidden spikes per sample (gentle; 658 vs. 390 gentler, 388 vs. 201 aggressive), the spike trains first differ in window 4 (median), and the class agrees for 90–96 % of samples. The gated network is not an approximation of an ungated one: removing the gate from the gentle weights lowers accuracy (93.5 vs. 94.8 % on these samples). Accuracy is therefore the measure of (B), and the gate is trained into the network for that reason.

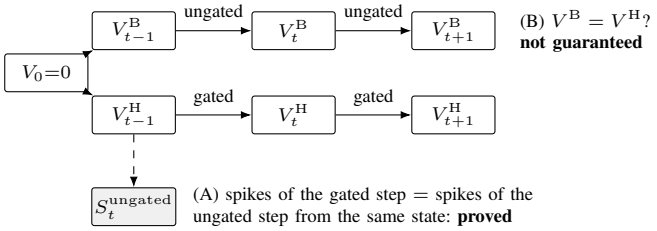


Fig. 1. Scope of the guarantee. Both runs start from the same $V_0 = 0$. (A) holds at every timestep; (B) does not, because sleeping neurons receive the mean-field drive. Measured on 1,000 samples: 0 spikes violate (A); the trajectories differ from window 4 (median) and the ungated run emits 1.8–1.9× as many hidden spikes.

We train with the gate in the loop: the forward pass uses the hard gate, and gradients flow through whichever update (exact or mean-field) it selected. Two penalties are added to the cross-entropy: the envelope's mean excess over the threshold, $\lambda_e \,\mathrm{mean}_{c,t} \max(0, E_c - \theta)/\theta$, and the expected positive proxy drive per timestep, $\lambda_d \,\mathrm{mean}_c \sum_j r_j \max(0, P_{jc})/\theta$ with r_j the firing rate of input j, which shrinks the largest weight of each sub-block. The three points use $(\lambda_e, \lambda_d) = (0.25, 0.1)$ gentler, (0.5, 0.25) gentle and (2, 1) aggressive; larger penalty weights put more sub-blocks to sleep at some cost in accuracy.

IV. Architecture

Hidden layer. 256 neurons form 8 blocks of 32 lanes (4 sub-blocks of 8). The weight memory is four 64-bit column RAMs, one per sub-block; a column is read only if its sub-block is awake. Per timestep the sequencer combines the envelope bank (gates), then visits each block: an awake block streams the timestep's events through a 4-stage pipeline (event read, index register, weight read, accumulate) and finalises its lanes in two stages; an asleep block takes 3 cycles (state read, mean-field finalise, write-back). The envelope unit for timestep $t+1$ runs during the block loop of t (double-buffered banks; Fig. 3). The cycle cost of a timestep with $n > 0$ events is $1 + \sum_\beta c_\beta + 2$ over the 8 blocks β, with $c_\beta = 4 + (n + 4)$ for an awake block and 3 for an asleep one, plus any wait for the envelope bank. Time is therefore saved per 32-neuron block (all four sub-blocks asleep), while energy is also saved per 8-neuron sub-block, whose weight column is not read.

Whole network. Hidden-layer spikes enter a 32-entry FIFO; a drain converts each set bit into an event of the output layer (32 lanes, 10 used, dense), which runs one timestep behind the hidden layer and overlapped with it. The class is the integer argmax of spike counts, ties broken by the summed pre-reset potential, then by index.

Stream front-end. Raw DVS events (x, y, p) are pooled 2×2 into the 578 inputs, de-duplicated per window (the model's input is a 0/1 raster), and written into a ring of 1,024 events (one RAMB18); an end-of-window marker closes the window, writes its range and lets the hidden layer start its envelope pass for that window. Markers have reserved FIFO

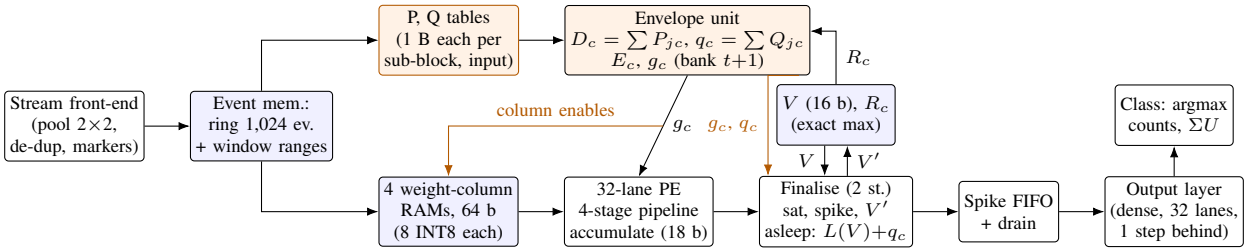


Fig. 2. HUSH whole-network datapath. The envelope unit computes the gates of timestep $t+1$ from the proxy and mean tables while the PE processes timestep t; a sub-block's weight column is read only when its gate is set, and a sleeping sub-block is finalised with the mean-field drive q_c. Blue: memories; orange: added by HUSH (the envelope unit, the P and Q tables, and the gate and q_c paths).

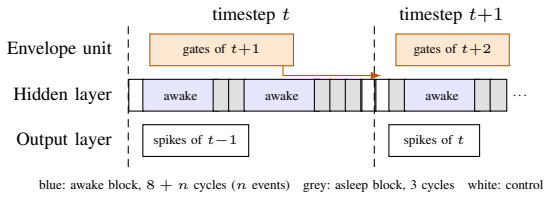


Fig. 3. Schedule of one timestep (not to scale). The eight 32-neuron blocks are visited in turn; the envelope unit prepares the gates of the next timestep meanwhile, and the output layer consumes the previous timestep's spikes.

space, so overload can only drop events. In stream mode the whole network takes 8,736 LUTs / 48.5 BRAMs (BASE) and 14,606 / 62.5 (HUSH), both at 100 MHz.

V. Verification

A golden integer model (Python) defines the behaviour. The RTL matches it bit for bit on 80 randomised layer configurations (gate, mean-field, sizes, leak, threshold, load port) and, for the whole network, on 50 N-MNIST test samples for each of four weight sets: every hidden and output block result and every class, with cycle counts equal to an exact cycle model. Forty random whole networks (including a 4-entry spike FIFO that forces back-pressure) are also bit-exact. The stream mode is bit-exact in every run without dropped events, at true sensor timing and up to 2,000× replay. The safety arithmetic is verified formally with SymbiYosys (ABC PDR) on the RTL modules used in the design, in two complementary checks. (1) At the implementation's widths (16-bit V, 18-bit accumulators, $k = 4$, $\theta = 64$; both the 578-input and the 256-input layer), one timestep of an 8-lane sub-block with every value free proves that, if each $V_i \le R_c$ and each neuron's drive is at most D_c: a sleeping sub-block contains no neuron whose ungated U reaches θ; every $U_i \le E_c$, awake or asleep; no sum wraps around (each 18-bit result equals the same sum computed 8 bits wider); and $R' = \max V'$ restores the hypothesis. With $V = R = 0$ at reset this is an induction over timesteps (72 s). The last property only checks that a maximum bounds its inputs; the RTL's max tree for R_c and the second finalise stage are covered by the simulations below. The 256-input run adds only the no-wrap check at the output layer's widths. (2) A sequential model at reduced widths proves the same properties over unbounded time for both sleep semantics.

TABLE I
Widths and the values that can occur (hidden layer, $M = 578$).

Quantity	Width	Range that can occur
W, P, Q	8 b	$[-128, 127]$
V, R ($L(V)$)	16 b	$[-32768, 32767]$ ($\lvert L(V)\rvert \le 30720$)
D_c, q_c (≤ 578 inputs)	18 b	$[-73984, 73406]$
Lane sum $L(V) + \sum W$	18 b	$[-104704, 104126]$ (18 b: ± 131072)
U, E	18 b	clipped to the 16-bit range
Class ΣU per output	32 b	$\lvert\Sigma U\rvert \le 3.28$ M

Mutants with an unsafe proxy (drive allowed above D_c) fail both checks immediately. The assumption on the drive is discharged by construction: the proxy and mean tables are generated offline from the weights and checked by the bit-exact simulations, which also cover memories and sequencing. Table I lists the bounds.

Gate-level simulation of the routed netlists reproduces the model's cycle count for every inference (+3 cycles of restart).

VI. Results

Setup. XC7A100T-1CSG324C, Vivado 2024.1, 100 MHz. Power is Vivado's estimate (`report_power`) annotated with the gate-level switching activity (SAIF) of 20 back-to-back test inferences on the routed netlist (99.9–100 % of nets matched, confidence High); it is not a board measurement. Energy per inference = power × latency while the chip is processing (*active* energy; Section VII discusses energy at sensor speed). All HUSH and BASE builds use Vivado's default synthesis and implementation directives. Accuracy on the full 10,000-sample N-MNIST test set ($T = 100$ windows of 3 ms) with the hardware's integer decision rule; accuracy and latency over three training seeds; area, power and energy for the seed-0 networks on 20 samples. BASE's power and energy were estimated with its own (22-epoch) weights; they equal those of the 10-epoch BASE (152 mW, 42.5 µJ). Training (Table II) uses the integer model of Section II in PyTorch and keeps the final epoch (no model selection on the test set); the trained weights are then evaluated in the exported integer model, which applies the 16-bit clip that the training model omits.

Accuracy–savings front. Table III and Fig. 4; speedups are relative to the ungated network of the same seed. The gentler and gentle networks are fine-tuned for 12 epochs from the 10-epoch ungated network, the aggressive one for

TABLE II
TRAINING SETUP (ALL POINTS, N-MNIST).

Input	N-MNIST [11], 34×34×2 events, 2×2 pooling → 578; $T = 100$ windows of 3 ms; 0/1 raster
Network	578-256-10 integer LIF, $k = 4$, $\theta = 64$, soft reset (16-bit clip only at evaluation)
Weights	INT8: round-and-clamp in the forward pass, straight-through gradient
Spike gradient	fast sigmoid [12], $1/(s(1 + \lvert x\rvert/s))^2$ with $s = \theta/4$
Readout loss	cross-entropy on $(\#\text{spikes} + 0.01\,\overline{U}/\theta) \cdot 25/T$
Regulariser	$5 \cdot \max(0, \bar{r} - 0.05)$ on the hidden firing rate $\bar{r}$
Optimiser	Adam [13], one-cycle schedule [14] (15 % warm-up), batch 128
BASE	10 epochs, peak lr 0.05; then 12 epochs from it, lr 0.02 (no gate)
HUSH	from 10-epoch BASE, lr 0.02, gate in the loop, λ_e, λ_d per point; 12 epochs (gentler, gentle), 6 epochs (aggressive)
Seeds	0, 1, 2 (weight init and batch order)

TABLE III
ACCURACY AND LATENCY, WHOLE NETWORK ON CHIP (MEAN ± STD, 3 SEEDS; LATENCY OVER 500 TEST SAMPLES).

Design	Acc. (%)	Reads[c]	Lat. (µs)	Speedup[e]
BASE (gate off)[b]	98.21±0.13	1.000	279.5	1.00×
HUSH gentler	96.54±0.07	0.204	159.8±8.1	1.75×
HUSH gentle	95.74±0.07	0.118	114.9±7.0	2.44×
HUSH aggressive[a]	92.32±0.68	0.070	82.9±7.8	3.39×

[a] fine-tuned for 6 epochs; the other gated points for 12. [b] trained as long as the gentler and gentle nets (10 + 12 epochs); 10 epochs only: 97.66±0.07. [c] weight-column reads relative to BASE; adding HUSH's own reads of P and Q (2 B per sub-block and input, vs. 8 B per column), its total weight-memory traffic is 0.32–0.45 of BASE's. [e] mean of the per-seed ratios.

6 (not re-run at 12); the same fine-tuning without the gate raises the ungated network from 97.66 % to 98.21 %, so we use that network as BASE. Against it the gate costs 1.7–5.9 points and reads 7–20 % of the weight columns (32–45 % of BASE's weight-memory bytes once its own P and Q reads are counted). Training longer with the gate (12 instead of 6 epochs) improved every axis of the gentle point (95.37 %, 1.98× → 95.74 %, 2.44×). In a single-seed test, a wider hidden layer (384) raised the ungated accuracy to 98.0 % but not the gated one (95.9 %, 6 epochs).

Same chip. Table IV gives the power builds; Table V reconciles them with the stream builds. A power build stores 20 test recordings on chip (53k events, ≈32.5 BRAM tiles) so that power can be simulated over 20 inferences without a host; a stream build instead has the sensor front-end and a 1,024-event ring. HUSH adds the same ≈6k LUTs and 14–15 BRAM tiles in both (envelope unit, P and Q tables, sub-block control): 79 % more LUTs and 19 % more BRAM than BASE in the power builds. The LUT×latency product therefore improves less than latency: by 3 % (gentler), 26 % (gentle) and 44 % (aggressive).

Spiker+ (clock-driven, ungated) runs BASE's 10-epoch INT8 weights (97.53 % on its neuron semantics, fixed point with 6 fractional bits), scans all 578 inputs serially each timestep and needs 81.8k cycles per inference in our harness

TABLE IV
HARDWARE ON XC7A100T, 100 MHz (WHOLE NETWORK, 20 ON-CHIP SAMPLES).

Design	LUT	BRAM	WNS (ns)	Lat. (µs)	Power (mW)	Energy (µJ)
BASE	7,745	80	+0.956	279.5	152	42.5
HUSH gentler	13,885	95	+0.450	151.2	157	23.7
HUSH gentle	13,885	95	+0.450	115.3	160	18.4
HUSH aggr.	13,885	95	+0.450	87.7	164	14.4
Spiker+ [1]	13,548[a]	70.5	+0.005[a]	818.4	311	254.5

Latency and power over the 20 simulated inferences; static power 101–102 mW.
[a] high-effort implementation; power measured on the default-effort netlist (WNS −0.241 ns).

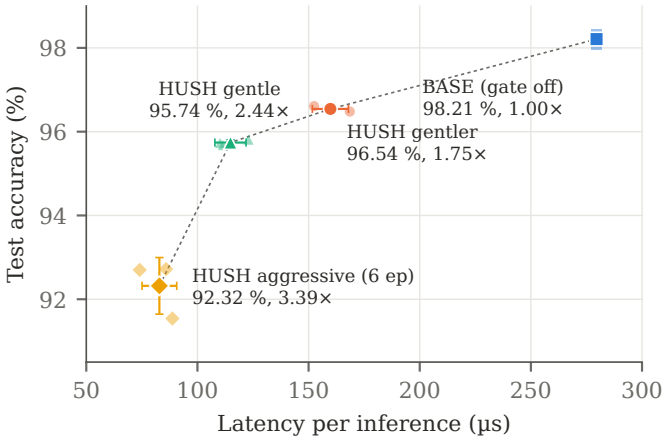


Fig. 4. Accuracy against latency, whole network on chip. Faint markers: the three seeds; solid: their mean ± std. Labels give accuracy and speedup over BASE.

(81.4k for the bare generated network). With default implementation settings it misses 100 MHz by 0.24 ns; with high-effort directives it meets 100 MHz (+0.005 ns, 13,548 LUTs). Its power was measured on the default-effort netlist; the high-effort one is estimated 4 % higher, so the reported Spiker+ energy is a lower bound. With the gate, HUSH spends 44–66 % less energy per inference than BASE although its power is 3–8 % higher: the extra envelope logic costs less than the time it saves; this is active energy per inference (Fig. 5). Two thirds of BASE's active energy is static power integrated over its latency, so most of the saving comes from finishing sooner; dynamic energy alone falls by 41–62 %. Spiker+ draws 311 mW, 140 mW of it in block RAM, which it reads every cycle (ours: 12 mW, read only for awake sub-blocks), and needs 6.0× the energy of BASE and 13.8× that of HUSH gentle. Energies in Table IV use the latency of the same 20 inferences (e.g. gentler 151.2 µs vs. 159.8 µs over 500 samples); the speedups of Table III use the 500-sample means.

Heuristic gates (exploratory). To show what the proof buys, we keep the HUSH-trained gentle network and swap only the predictor, charging each its own memory reads: HUSH 2 B (P, Q) per sub-block and active input; a state-

TABLE V
RESOURCES OF THE FOUR BUILDS (BRAM IN 36 KB TILES).

Build	Hidden layer		Output	Glue[f]	Total	
	LUT	BRAM	LUT/BRAM	LUT	LUT	BRAM
BASE, power	3,874	74	3,117/6	754	7,745	80
HUSH, power	9,922	89	3,150/6	813	13,885	95
BASE, stream	4,482	41.5	3,142/7	1,112	8,736	48.5
HUSH, stream	10,365	56.5	3,122/6	1,119	14,606	62.5

Power builds hold 20 recordings on chip (Table IV); stream builds hold a 1,024-event ring and the sensor front-end (Section IV). [f] spike FIFO, drain, classifier, front-end and board harness (UART loader, LED fold); total minus the two layers. All four meet 100 MHz (WNS +0.252 to +0.956 ns).

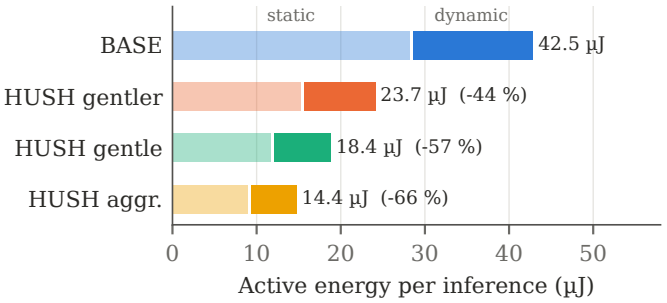


Fig. 5. Active energy per inference (Table IV), split into static power × latency (light) and dynamic energy (dark). Spiker+ (254.5 µJ, 171.9 dynamic) is off this scale.

TABLE VI
SWAPPING THE PREDICTOR (GENTLE NETWORK, 1,000 SAMPLES).

Matched on	Predictor	Acc. (%)	Reads	Traffic	Suppr. spikes
–	**HUSH**	**94.8**	0.105	0.355	**0**
reads	Early-style	92.4	0.103	0.350	485
reads	SITRA-inspired	68.1	0.102	0.227	117,450
traffic	Early-style	93.6	0.116	0.359	409
traffic	SITRA-inspired	87.4	0.279	0.404[g]	74,095

Reads and traffic relative to BASE. [g] integer threshold; the next step gives 0.23.

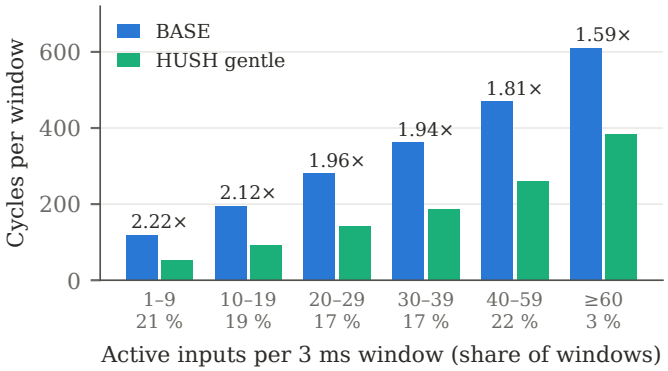


Fig. 6. Cycles per 3 ms window against the number of active inputs in it (cycle model, 500 test recordings, gentle network), with the speedup of HUSH above each pair.

free proxy inspired by SITRA (count of negative weights among the active inputs; not SITRA's MSB dot product) 1 B of sign bits; an Early-style predictor ($L(V)$ plus the 8 largest active weights) the 8 weight columns it needs before deciding. Thresholds are bisected to HUSH's column reads or total traffic (Table VI, 1,000 samples, seed-0 network). Matched on traffic, the Early-style predictor loses 1.2 points and suppresses 409 spikes; its charge is a lower bound (the 8 neurons of a sub-block may need different top-8 sets), so its true traffic is higher. The state-free one cannot be matched exactly (integer threshold): at the nearest step it moves 14 % more bytes than HUSH and still loses 7.4 points. The networks were trained for HUSH, not for the heuristics, so this measures the predictors, not the best network each could reach.

VII. REAL-TIME OPERATION

N-MNIST recordings deliver on average 41.9 raw events per 3 ms window (26.5 distinct pooled inputs; maximum 96). Real-time feasibility is shown by cycle-accurate RTL replay of N-MNIST event streams at their recorded timing, not by a sensor connected to the board. **At true sensor timing** (4 recordings per design, gentle network, bit-exact, no drops) a window is processed in 1.47 µs on average with HUSH (3.04 µs BASE), at most 4.50 µs (6.64 µs), i.e. ≤ 0.22 % of the window; the class is ready 0.89 µs (2.03 µs) after the last window; the input ring never holds more than 127 events. HUSH halves the average window (1.47 vs. 3.04 µs, −52 %) but shortens the busiest one less, because most blocks wake up in it (Fig. 6): over 500 test recordings, windows with 1–9 active inputs run 2.2× faster, windows with 60 or more 1.59×. **Energy in real-time operation.** The 44–66 % saving of Section VI is active energy per inference. At sensor speed the chip idles more than 99.9 % of the time, so a 300 ms recording costs ≈30.3 mJ of static energy (101 mW) for either design, and HUSH saves only its dynamic share (6.7 vs. 14.3 µJ per inference), i.e. < 0.03 % of the always-on total. HUSH's gain appears when the chip is kept busy, e.g. serving recordings faster than real time. At 750×, the highest replay speed at which BASE drops nothing on the heaviest recordings, a recording occupies 0.40 ms; static power over that time plus the dynamic energy of one inference gives 101 mW×0.40 ms + 14.3 µJ = 54.7 µJ for BASE and 102 mW×0.40 ms + 6.7 µJ = 47.5 µJ for HUSH gentle (−13 %); HUSH can also run at 1,000× (37.3 µJ). **Faster than real time.** Replaying the 20 test recordings with the most events (1.7× the mean rate) with the same 1,024-event buffer, at speeds in steps of 250×, BASE drops events above 750× real time, HUSH gentle above 1,000× on all 20 recordings (23.7 M events/s, 1.33×) and above 1,500× for the median recording (2.0×); HUSH gentler also holds 750× on all 20 and 1,000× on 18 (Fig. 7). **Early decisions.** Because the class can be read after any window, Fig. 8 reports accuracy vs. elapsed sensor time. Between 21 ms and 156 ms the gentler network is more accurate than BASE trained equally long (e.g. 87.7 % vs. 79.5 % at 60 ms and 92.6 % vs. 88.9 % at 99 ms); BASE leads from 159 ms on. The gated networks do not spike earlier or more (fewer output spikes in the first 99 ms: 39–41 vs. 52) but their early leading class is wrong

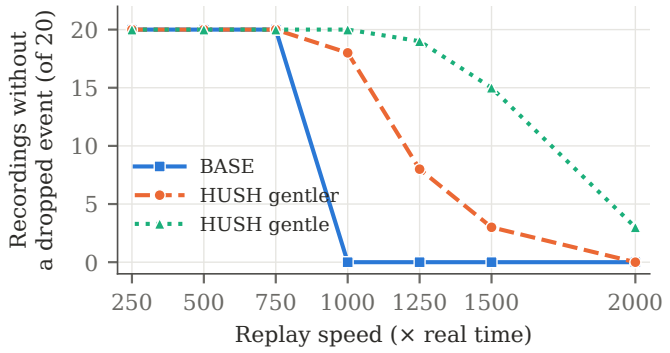


Fig. 7. Replay faster than real time: how many of the 20 heaviest test recordings are processed bit-exactly with no dropped event (RTL, same 1,024-event buffer).

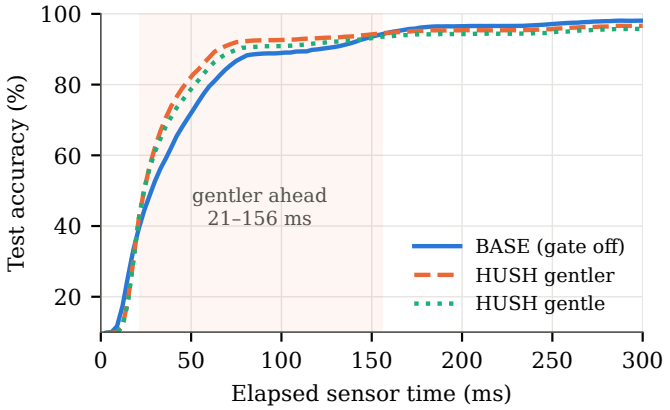


Fig. 8. Accuracy when the class is read after the first windows of a recording (seed-0 networks, 10,000 test samples). BASE is trained as long as the gentler and gentle networks.

less often (8.5–8.8 % vs. 10.6 % of 1,000 samples at 99 ms). The gentle network leads from 21 ms to 144 ms. An application that decides between 21 and 144 ms is therefore both faster and more accurate with either gated network. (Spike statistics: first 1,000 test samples.)

VIII. Comparison with the State of the Art

HUSH runs 37–435× faster per inference than the designs that report latency and needs 77–950× less active energy per inference (18.4–23.7 µJ vs. ≈1.83 and ≈17.5 mJ computed by us as their power × latency), on a smaller device with 3.8–5.6× fewer LUTs than Cheng et al. and FeNN-DMA and 2.7–11.3× fewer than Wu et al.'s range. Accuracy is comparable to those two designs (96.54 vs. 96.4 and 95.76 %) and 1.6–1.9 points below the two larger designs; with the gate off, the same hardware reaches 98.21 % at 280 µs. These rows are not like for like. *Networks*: Cheng et al. use a structured-sparse event-driven SNN, Wu et al. an asynchronously triggered one, FeNN-DMA a 512-neuron recurrent network, the Surrogates design a VGG9-style convolutional SNN; ours is 578-256-10. *Devices*: all others use UltraScale+ parts with 1.8–19× the LUT capacity of the XC7A100T. *Latency*: ours is the compute time of one 300 ms recording whose events are already on chip; the others report their own per-sample processing time. *Power*: ours and Cheng et al.'s are Vivado estimates from simulated activity, FeNN-DMA's is a vectorless estimate with the processing system removed; the Surrogates and Wu et al. figures are as reported, and their energies are computed by us. Only the same-chip comparison (Tables III–V) isolates HUSH's own effect.

TABLE VII
FPGA SNN accelerators on N-MNIST.

Design	FPGA	Acc. (%)	LUT	Power	Latency
Cheng et al. [9]	ZCU104	98.11	81.3k	0.81[d]	n/a
FeNN-DMA [15]	KV260	98.46	55.0k	0.53[d]	n/r
Surrogates [16]	xcvu9p	96.4	n/r	0.31	5.9 ms
Wu et al. [17]	ZCU102	95.76	39.7–165k	0.33–0.37	50 ms
HUSH gentler	XC7A100T	96.54	14.6k	0.16/0.056[d]	0.16 ms
HUSH gentle	XC7A100T	95.74	14.6k	0.16/0.058[d]	0.11 ms

Power in W; [d] dynamic only; n/r: not reported; n/a: not available to us. HUSH LUTs: stream build (Table V).

IX. Limitations and Conclusion

The evaluation covers one dataset and one small fully connected network with a single gated layer and one sub-block size. The speedup depends on stream sparsity: per window it falls from 2.2× with 1–9 active inputs to 1.6× with 60 or more, as more sub-blocks wake up, and it tends to 1× on dense streams. Serving many streams on one chip is limited by state memory, where BASE would fit more streams than HUSH. The gate costs accuracy (1.7–5.9 points) and area (79 % more LUTs); its guarantee is per timestep, not for the whole trajectory; power is estimated from simulated activity, not measured on a board; and real time is shown by RTL replay at sensor timing, not with a live sensor. HUSH shows that output-side skipping in SNN hardware can be both effective and provably safe; extending the envelope to convolutional layers and denser event streams (DVS-Gesture) is future work.

Code availability. The RTL, golden model, formal harnesses (SymbiYosys [18]), training and analysis scripts and all result files will be released with the paper.

References

[1] A. Carpegna, A. Savino, and S. Di Carlo, "Spiker+: a framework for the generation of efficient spiking neural networks FPGA accelerators for inference at the edge," arXiv:2401.01141, 2024.

[2] ——, "Spiker: an FPGA-optimized hardware acceleration for spiking neural networks," arXiv:2201.06993, 2022.

[3] J. Li, G. Shen, D. Zhao, Q. Zhang, and Y. Zeng, "FireFly: A high-throughput hardware accelerator for spiking neural networks with efficient DSP and memory optimization," arXiv:2301.01905, 2023.

[4] X. Zhang, Z. Song, P. Zhou, X. Li, X. Liu, X. Lin, Z. He, L. Jiang, N. Jing, and X. Liang, "Early: an importance-aware early firing and exit for SNN acceleration," in *Proc. IEEE Int. Conf. Computer Design (ICCD)*, 2024.

[5] A. Bhattacharjee, A. Moitra, R. Yin, and P. Panda, "SITRA: Exploiting temporal silence in spiking transformers for fast & energy-efficient inference," in *Proc. IEEE/ACM Int. Symp. Low Power Electronics and Design (ISLPED)*, 2025, pp. 1–7.

[6] Z. Wang, F. Liu, X. Tang, and L. Jiang, "PS4: A low power SNN accelerator with spike speculative scheme," in *Proc. IEEE 42nd Int. Conf. Computer Design (ICCD)*, 2024, pp. 76–83.

[7] S. Matinizadeh, M. L. Varshika, and A. Das, "Optimizing memory latency and bandwidth of spiking neural network accelerators on FPGA via sparse hashing (SPARSH)," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2025, pp. 1–9.

[8] K. Lee and J. Park, "C-STEP: Compute-efficient spiking transformers with temporal exit and early-guided pruning," in *Proc. Design, Automation and Test in Europe (DATE)*, 2026.

[9] X. Cheng, S. Cao, S. Wang, M. Wang, W. Li, and X. Zeng, "An FPGA-based event-driven SNN accelerator for DVS applications with structured sparsity and early-stop," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 7, pp. 3298–3310, 2025.

[10] V. Akhlaghi, A. Yazdanbakhsh, K. Samadi, R. K. Gupta, and H. Esmaeilzadeh, "SnaPEA: Predictive early activation for reducing computation in deep convolutional neural networks," in *Proc. ACM/IEEE 45th Int. Symp. Computer Architecture (ISCA)*, 2018, pp. 662–673.

[11] G. Orchard, A. Jayawant, G. K. Cohen, and N. Thakor, "Converting static image datasets to spiking neuromorphic datasets using saccades," *Frontiers in Neuroscience*, vol. 9, 2015.

[12] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[13] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[14] L. N. Smith and N. Topin, "Super-convergence: very fast training of neural networks using large learning rates," in *Proc. SPIE 11006, Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, 2019.

[15] Z. Aizaz, J. C. Knight, and T. Nowotny, "FeNN-DMA: A RISC-V SoC for SNN acceleration," arXiv:2511.00732, 2025.

[16] I. Aliyev, J. Lopez, and T. Adegbija, "Surrogates, spikes, and sparsity: Performance analysis and characterization of SNN hyperparameters on hardware," arXiv:2603.24891, 2026.

[17] Z. Wu, M. Song, and H. K.-H. So, "Towards asynchronously triggered spiking neural network on FPGA for event-based vision," in *Proc. Int. Conf. Field-Programmable Technology (ICFPT)*, 2023, pp. 292–293.

[18] YosysHQ, "SymbiYosys: front-end for Yosys-based formal verification flows," https://github.com/YosysHQ/sby, 2026.